

TABLE DES MATIÈRES

Arbres AVL	3
Rappel :	3
Définition : Arbres AVL ou arbre H-équilibré	6
Exemple : Arbre AVL	6
Définition : facteur d'équilibrage	7
Exemples : Arbres AVL	8
Exemples : Arbres H-Equilibré et Non AVL	9
Rappel : intérêt des AVL	11
Propriété	12
Preuve	13
Techniques d'équilibrage	14
Principe d'Insertion dans un AVL	15
Remarque :	16
Exemple : insertion de 15	16
Rotations	18
Définition : rotation simple (Droite & Gauche)	19
Exemples	20
Exemples	21
Exemple	23
Remarque	23
Comment choisir la rotation à effectuer :	24
Principe de l'Algorithme	24
<i>Propriétés</i>	25
Exemple de construction d'un AVL	27
Implémentation d'un AVL	29
Implémentation des rotations	36
Fonction rotationG : AVL → AVL	37
Fonction rotationD : AVL → AVL	38
Double Rotation : Gauche+Droite	39
Double Rotation : Droite + Gauche	40
Propriétés des rotations :	41
<i>Insertion dans un AVL</i>	42
Principe :	42

Fonction équilibrage	43
Suppression dans les AVL	44
Propriété	44
Exemples de suppressions	45
Algorithme de suppression d'un élément dans un AVL	51
Algorithme de suppression du nœud racine dans un AVL	52
Algorithme de suppression du maximum dans un AVL	53
Complexité	54
Références	55

ARBRES AVL

Rappel :

Les arbres binaires de recherche proposent une organisation assez forte de l'ensemble des éléments.

Il est aisé dans un ABR de retrouver :

Le minimum = est le nœud le plus à gauche.

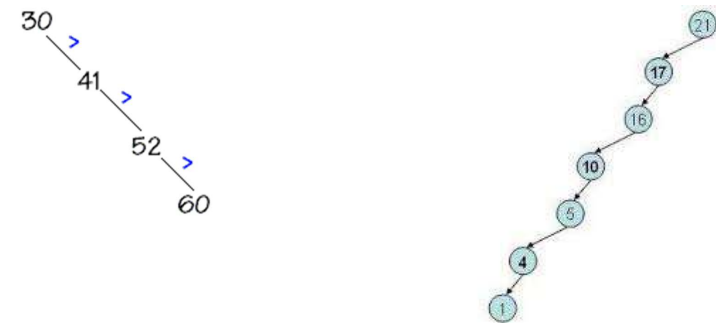
Le maximum = est le nœud le plus à droite.

La liste triée des éléments en effectuant le parcours infixe de l'arbre.

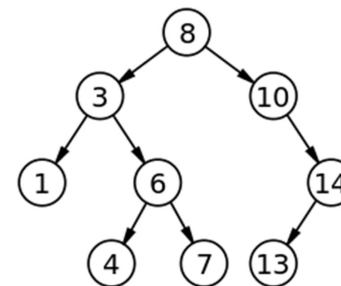
Les temps de calcul de ces opérations sont proportionnels à la hauteur de l'arbre, qui, lui-même, si on fait attention, est presque proportionnel à $\log_2(n)$ avec n le nombre de nœuds.

Les opérations de recherche, d'insertion et suppression dans un arbre binaire de recherche de n éléments se font en moyenne en $\Theta(\log_2(n))$ comparaisons.

Mais dans le pire des cas, la complexité de ces opérations est en $\Theta(n)$: un arbre binaire de recherche peut être dégénérer en une liste.



L'arbre binaire de recherche idéal pour lequel une recherche nécessite au pire $\log_2(n)$ comparaisons est un arbre complètement équilibré, c'est à dire un arbre dont les feuilles sont situées au plus sur deux niveaux.



Arbres équilibrés

Les structures d'arbres binaires de recherches permettent de réaliser des opérations dynamiques, telles que recherche, prédécesseur, successeur, minimum, maximum, insertion, et suppression en temps proportionnel à la hauteur de l'arbre.

- Si l'arbre est équilibré, la hauteur est en $O(\log(n))$ où n est le nombre de nœuds.

- Divers types d'arbres équilibrés sont utilisés :

- arbres AVL,
- arbres 2-3-4
- bicolores,
- b-arbres.

Arbre AVL

La première classe d'arbres équilibrés a été proposée dans les années 60 par G.M. Adelson-Velsky et E.M. Landis (d'où son nom)

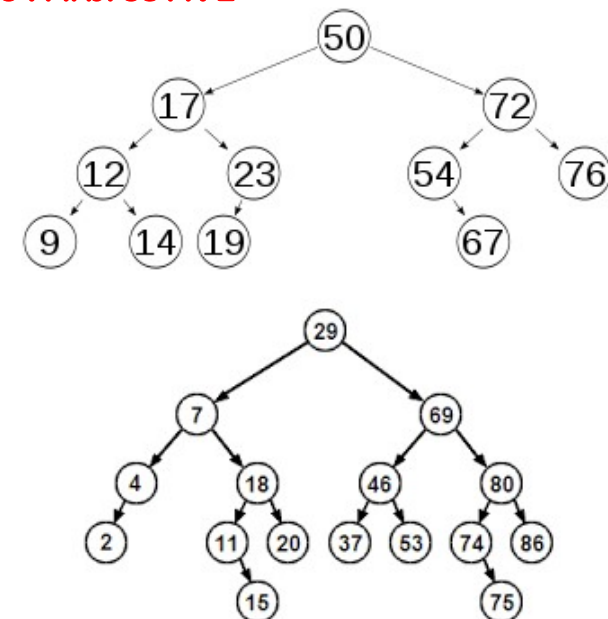
L'intérêt d'un arbre AVL se situe dans la recherche.

En effet, puisque l'arbre est équilibré, on évite d'avoir des situations où la recherche est spécialement longue.

Définition : Arbres AVL ou arbre H-équilibré

Un arbre binaire de recherche est un arbre AVL si, pour tout nœuds, les hauteurs des sous arbres gauche et droit diffèrent d'au plus 1 en valeur absolue.

Exemple : Arbres AVL



Définition : facteur d'équilibrage

Soit la fonction déséquilibre sur les arbres binaires, définie récursivement par :

Déséquilibre Arbre vide = 0, et

Déséquilibre $A = \langle R, G, D \rangle = h(D) - h(G)$

avec $h(\text{arbre vide}) = -1$ // *Par convention*

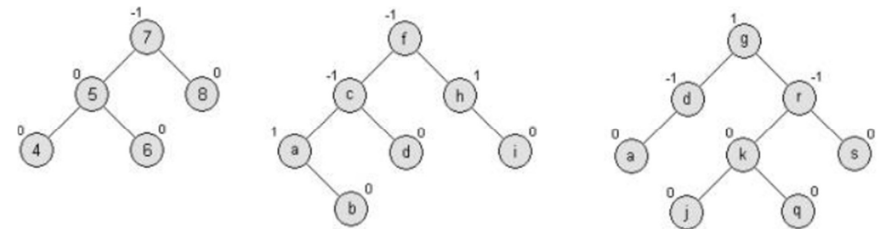
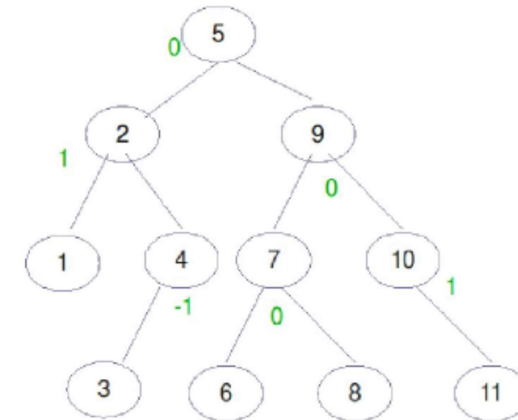
et si A est un arbre non vide : $h(A) = \text{hauteur}(A) + 1$.

Chaque nœud contient un entier appelé **facteur d'équilibrage** qui permet de déterminer s'il est nécessaire de rééquilibrer l'arbre.

Le facteur d'équilibrage d'un nœud dans un arbre AVL vaut 0, 1 ou -1.

C'est à dire pour tout nœud **a** de l'arbre A on a :

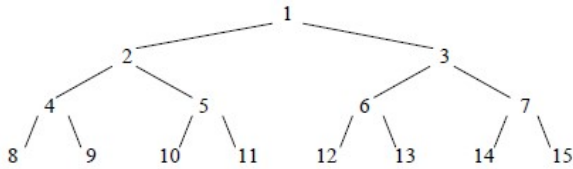
Déséquilibre (a) = $\{-1, 0, 1\}$



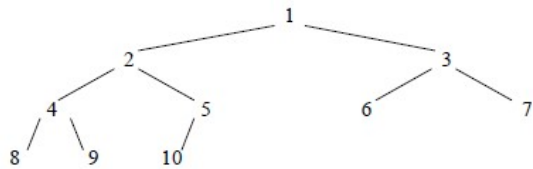
Exemples : Arbres AVL

Exemples : Arbres H-Equilibré et Non AVL

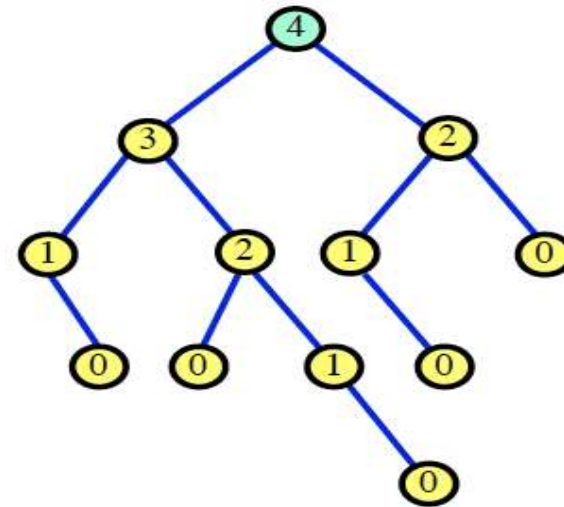
Arbre parfait complet



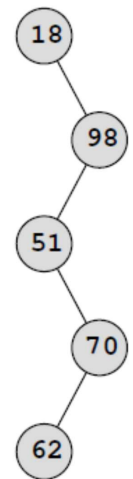
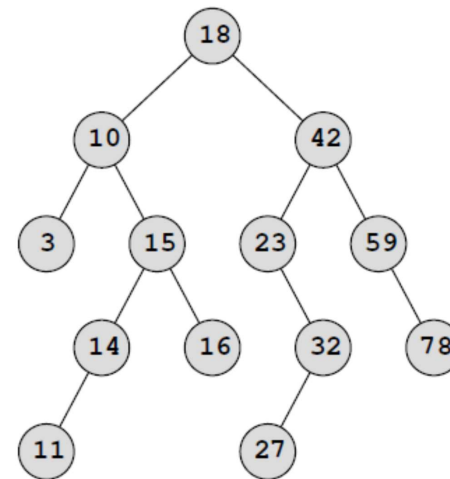
Arbre Parfait



b) H-Equilibré



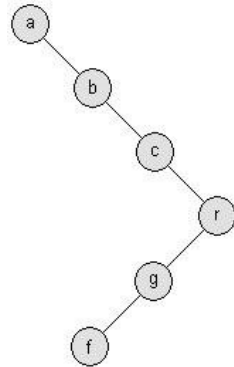
c) Arbre non H-équilibré



Rappel : intérêt des AVL

L'intérêt d'un arbre AVL se situe dans la recherche.

En effet, puisque l'arbre est équilibré, on évite d'avoir des situations où la recherche est spécialement longue (comme c'est le cas dans l'arbre suivant si on cherche « f »).



ABR non équilibré

Remarques :

Il existe des algorithmes de rééquilibrage après insertion ou suppression qui maintiennent la propriété d'AVL en effectuant une suite de rotations sur un chemin de la racine à une feuille de l'arbre.

Évidemment, on ne gagne jamais rien sans payer un prix. De façon à pouvoir chercher dans un arbre équilibré, il faut conserver l'équilibre lors des ajouts.

L'insertion (et la suppression) est plus lente dans un arbre AVL que dans un simple ABR.

Mais, les opérations de recherche, d'insertion et suppression ont une complexité qui reste logarithmique dans le pire des cas.

Propriété

Tout arbre H-équilibré ayant n nœuds et de hauteur h vérifiant la propriété suivante :

$$\log_2(1+n) \leq 1+h \leq \alpha \log_2(2+n) \text{ avec } \alpha \leq 1,44.$$

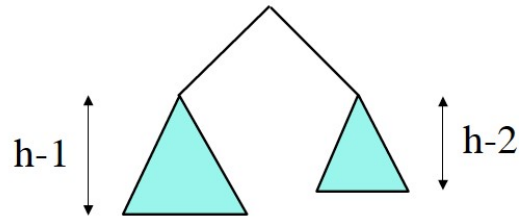
Par exemple, si $n=100000$, $17 \leq h \leq 25$.

Cette propriété montre que les contraintes de déséquilibre en chaque nœud permettent de limiter la hauteur totale de l'arbre : la hauteur d'un arbre H-équilibré de n nœuds est toujours de l'ordre de $\log_2(n)$.

Preuve

On a toujours $n \leq 2^{h+1} - 1$

Donc $\log_2(1+n) \leq 1+h$.



Soit $N(h)$ le nombre *minimum* de nœuds d'un arbre AVL de hauteur h .

Alors $N(h) = 1 + N(h-1) + N(h-2)$.

La suite $F(h) = N(h)+1$ vérifie

$F(0) = 2, F(1) = 3, F(h+2) = F(h+1) + F(h)$

Donc $F(h) = 1/\sqrt{5} (\phi^{h+3} - \phi^{-(h+3)})$

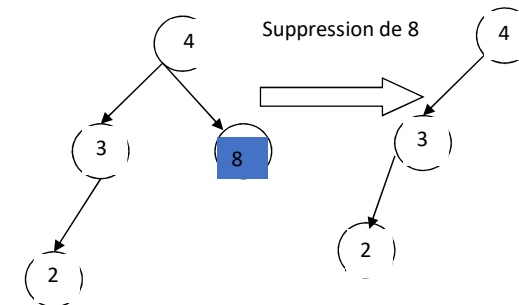
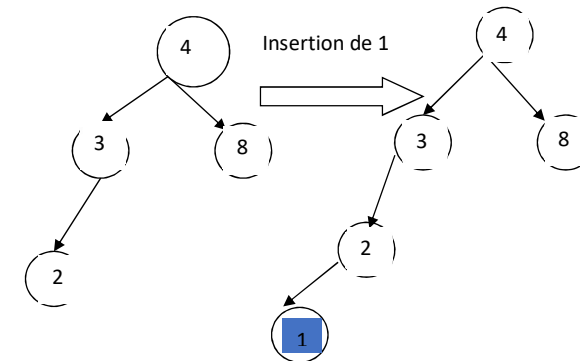
où $\phi = \frac{1+\sqrt{5}}{2}$ est le nombre d'or

$1+n \geq F(h) > 1/\sqrt{5} (\phi^{h+3} - 1)$

$h+3 < \log_2(2+n) / \log_2 \phi + \log_2 \sqrt{5} \leq \alpha \log_2(2+n) + 2$

TECHNIQUES D'ÉQUILIBRAGE

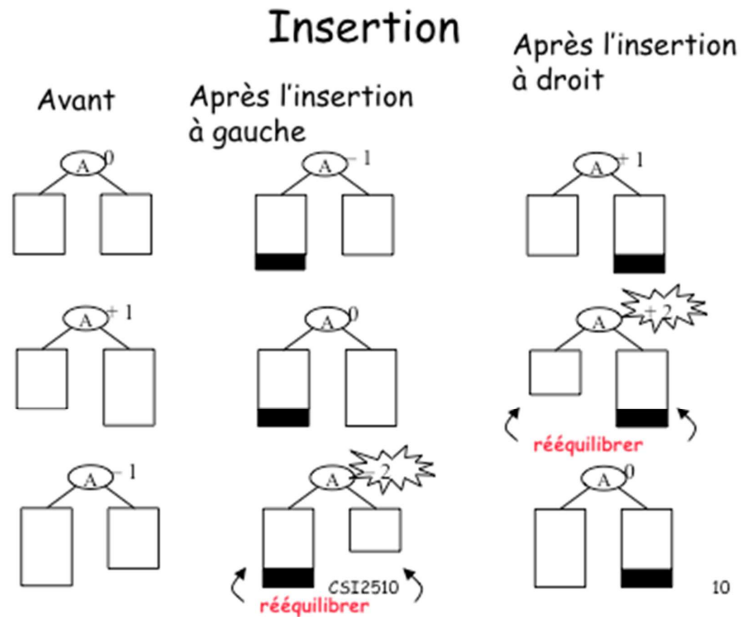
Une insertion ou une suppression dans un AVL peuvent déséquilibrer l'arbre :



Ainsi après avoir ajouté aux feuilles ou supprimé un élément dans un AVL, il faut éventuellement le rééquilibrer, tout en conservant la structure d'arbre binaire de recherche.

Principe d'Insertion dans un AVL

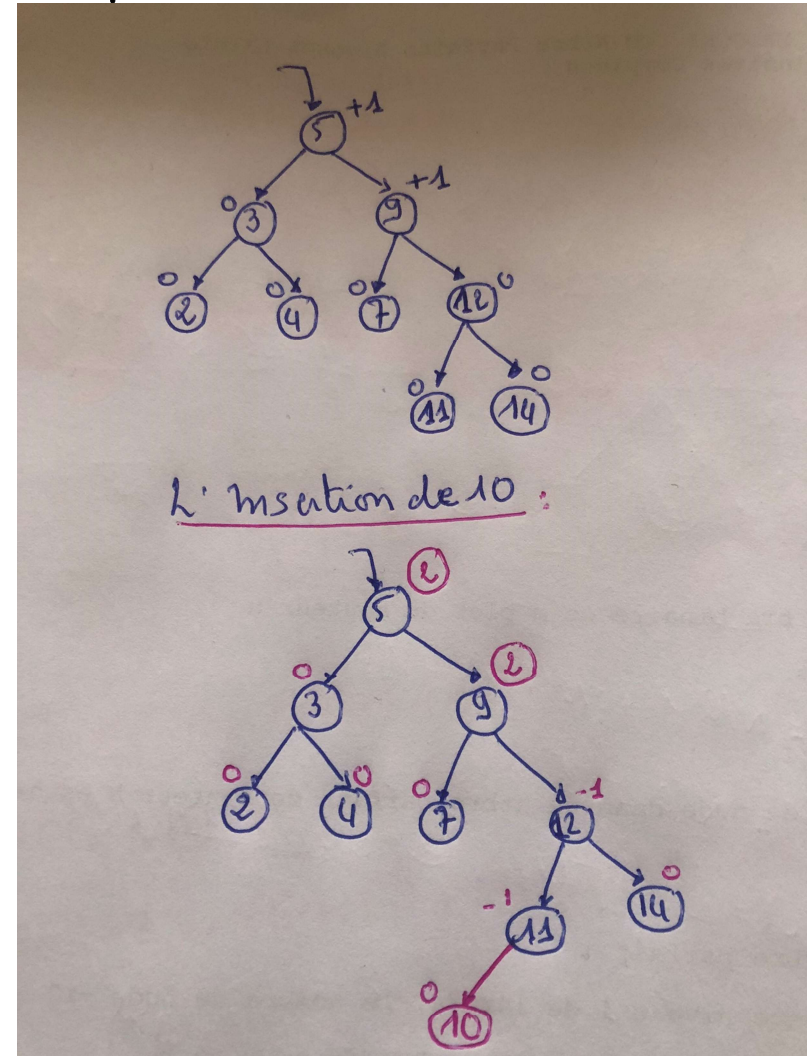
Les opérations d'insertion sont celles d'un arbre binaire de recherche dans lesquelles on ajoute la gestion du facteur d'équilibrage.



Remarque :

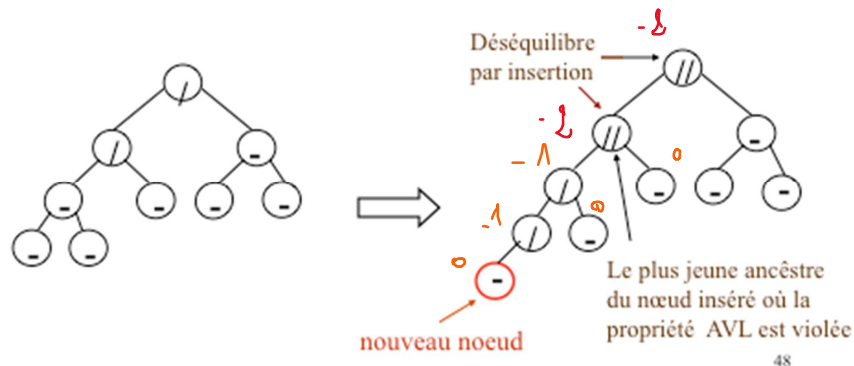
L'insertion d'un élément peut provoquer le déséquilibre de plusieurs nœuds

Exemple : insertion de 10



Arbres AVL: Insertion

Noter que l'insertion d'un nœud peut provoquer des déséquilibres sur plusieurs nœuds.



Remarques

Le déséquilibre maximal après une insertion est ± 2 .

Il est clair que l'insertion d'un élément « elt » ne peut entraîner le déséquilibre des nœuds du chemin de la racine à l'élément « elt », il suffit de rééquilibrer l'arbre à partir d'un seul nœud « n » (dans le chemin de la racine à l'élément « elt », « n » étant le dernier nœud pour lequel le déséquilibre est ± 2)

Rotations

Lors des insertions ou suppressions, l'arbre peut se *déséquilibrer* (valeur 2 ou -2), on utilise alors des **rotations** pour rééquilibrer l'arbre.

Équilibrer un arbre binaire de recherche A, revient à effectuer des opérations de rotations simples ou doubles. Ce qui permet de diminuer la hauteur de 1.

On distingue quatre types de rotation :

1) Rotations simples :

- ✓ Rotation Gauche
- ✓ Rotation droite

2) Rotations doubles :

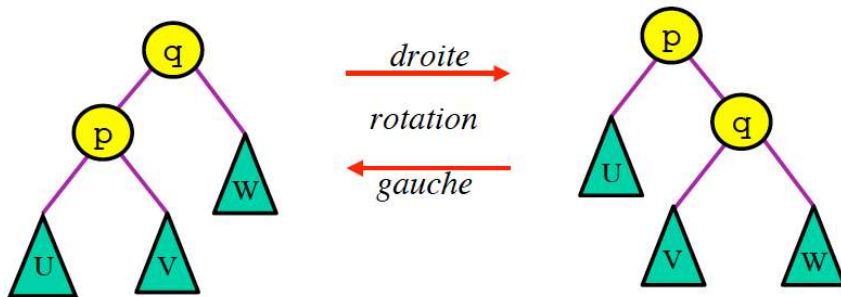
- ✓ Rotation gauche droite
- ✓ Rotation Droite Gauche.

Définition : rotation simple (Droite & Gauche)

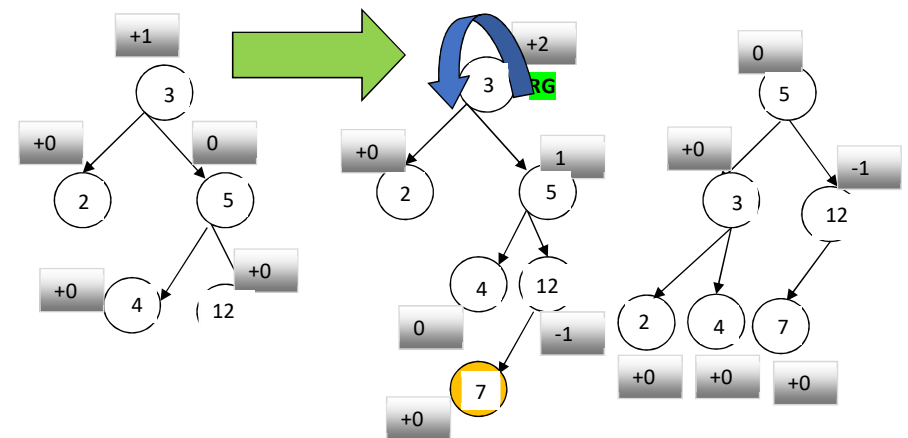
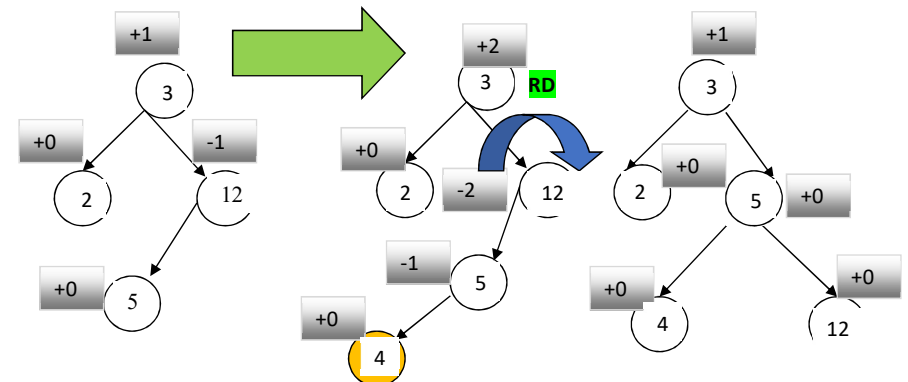
Une **rotation droite** autour du sommet « q » d'un arbre binaire de recherche consiste à faire descendre le sommet « q » et à faire remonter son fils gauche « p » sans invalider l'ordre des éléments. L'opération inverse s'appelle **rotation gauche** autour du sommet p .

Les **rotations** gauche et droite transforment un arbre

- Elles préservent l'ordre infixe
- Elles se réalisent en temps constant.

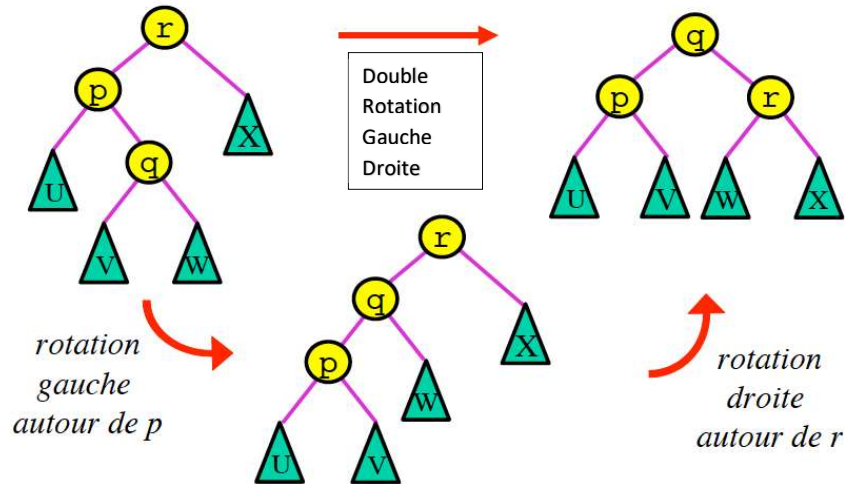


Exemples de Rotations

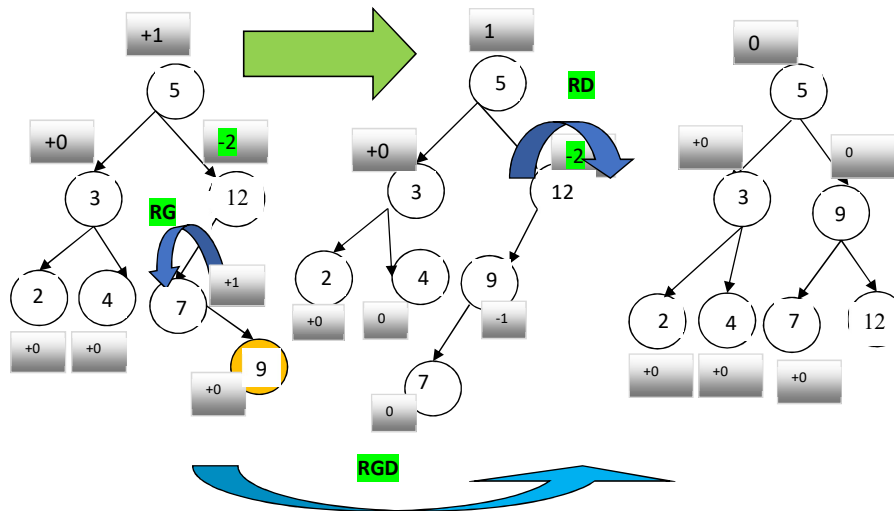


Définition Double rotation Gauche-Droite

La rotation gauche - droite sur l'arbre A est composée d'une rotation gauche sur le sous arbre gauche suivie d'une rotation droite sur A

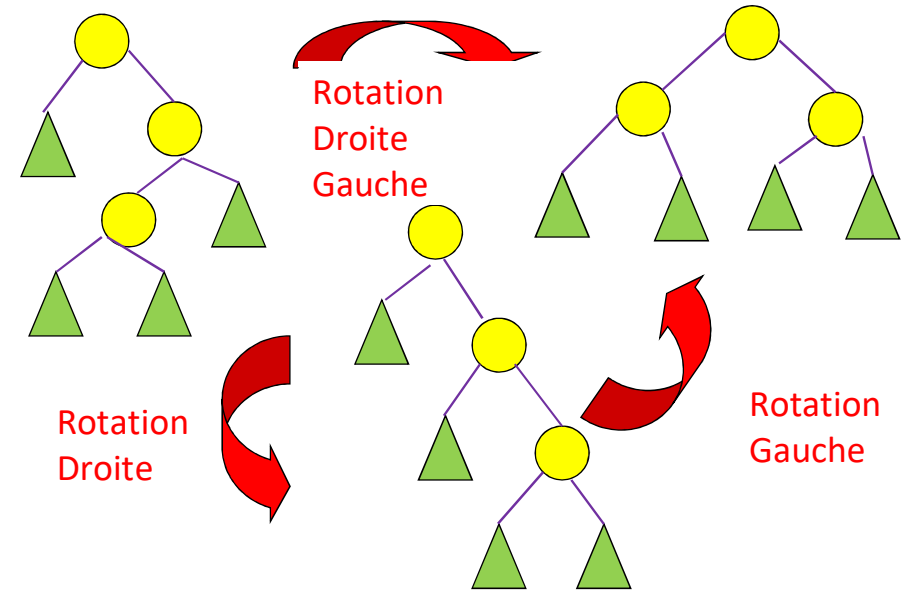


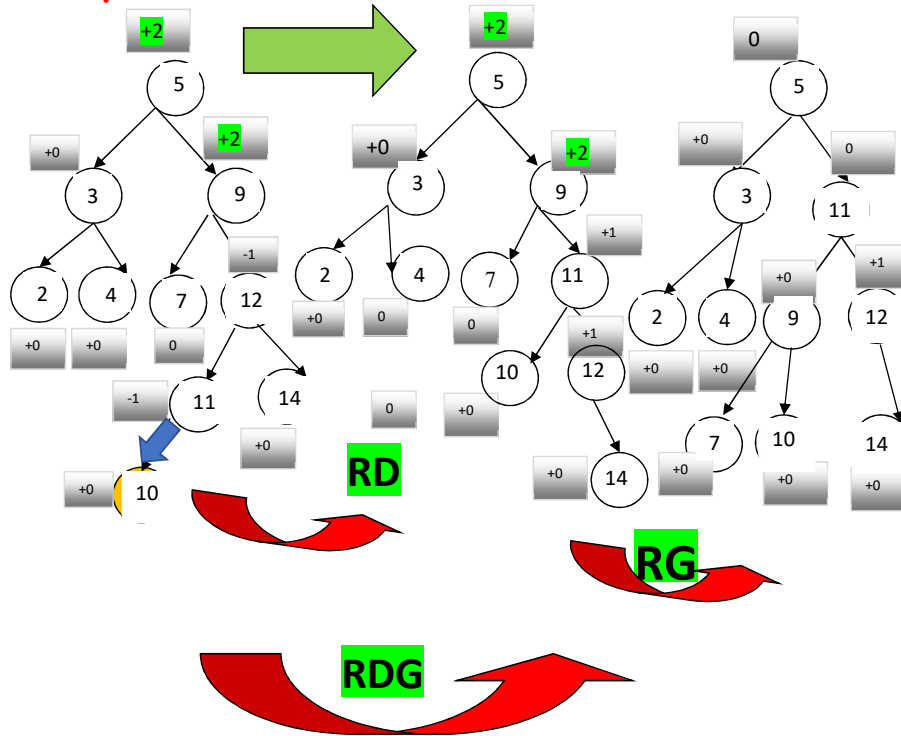
Exemples



Définition : Double rotation Droite - Gauche

La rotation droite - gauche sur l'arbre A est composée d'une rotation droite sur le sous arbre droit suivie d'une rotation gauche sur A



Exemple**Remarque**

Les opérations de rotations conservent la propriété d'arbre binaire de recherche si les éléments sont tous distincts.

Comment choisir la rotation à effectuer :

On utilise le facteur d'équilibrage

Soit Le nœud $A = \langle r, G, D \rangle$ et

Soit $h(A)$ la hauteur du nœud A .

Principe de l'Algorithme

Facteur d'équilibrage = Déséquilibre (A) = $h(D) - h(G)$

❖ Si Déséquilibre (A) = 0 (ou 1 ou -1) → Rééquilibrer(A) = A

Rotation simple :

❖ Si Déséquilibre (A) = -2 et

(Déséquilibre (G) = -1 ou Déséquilibre (G) = 0) →

Rééquilibrer(A) = **Rotation Droite** (A)

❖ Si Déséquilibre (A) = +2 et

(Déséquilibre (D) = +1 ou Déséquilibre (D) = 0) →

Rééquilibrer(A) = **Rotation Gauche** (A)

Rotation double :

❖ Si Déséquilibre (A) = -2 et Déséquilibre (G) = +1 →

Rééquilibrer(A) = **double Rotation GD** =

Rotation Gauche (G) puis Rotation droite (A)

❖ Si Déséquilibre (A) = +2 et Déséquilibre (D) = -1 →

Rééquilibrer(A) = **double Rotation DG** =

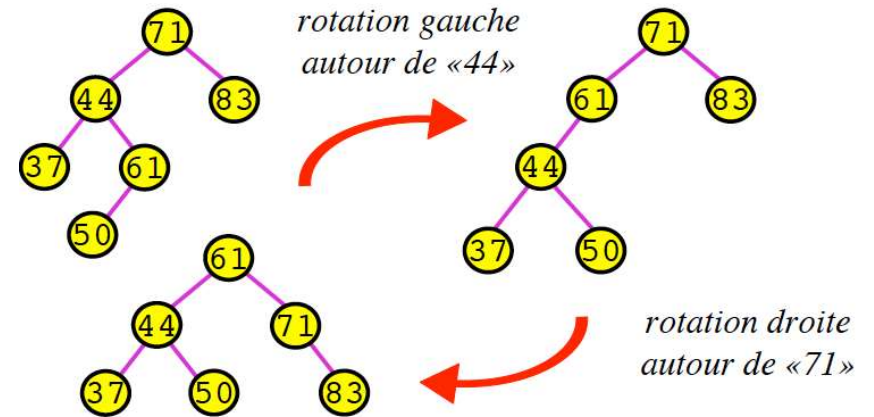
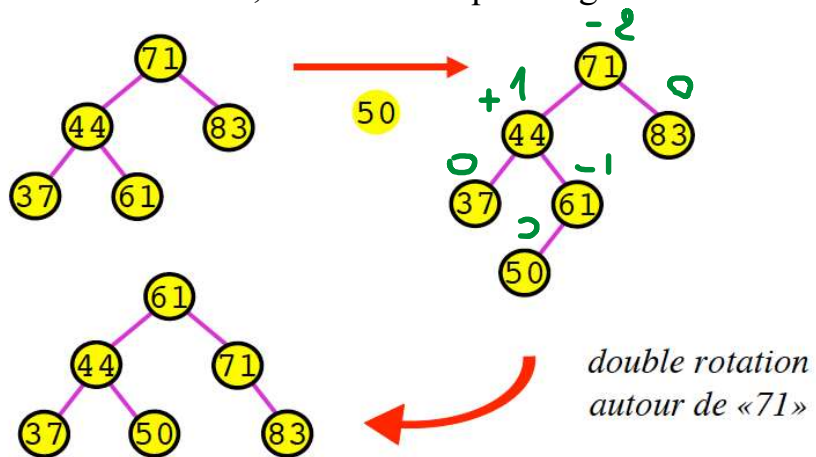
Rotation Droite(D) puis Rotation Gauche(A)

Propriétés

Pour rééquilibrer un arbre AVL après une **insertion**, une simple ou double rotation suffit.

Exemple d'Insertion dans un arbre AVL

Insertion ordinaire, suivie de rééquilibrage.

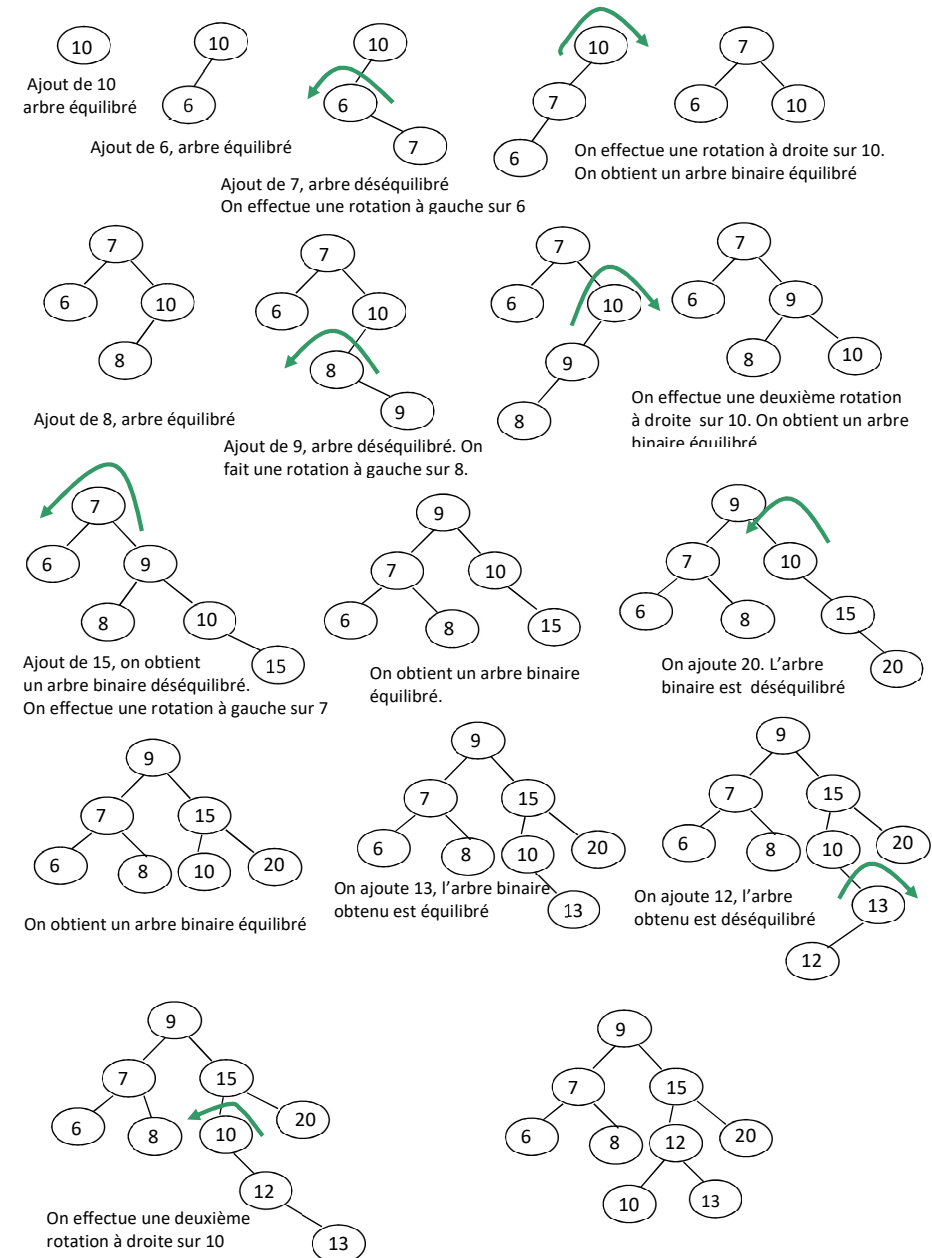


Exemple de construction d'un AVL

Construire un AVL à partir de la liste d'entiers suivante :

[10, 6, 7, 8, 9, 15, 20, 13, 12]

Et en rééquilibrant l'arbre après chaque insertion qui l'a déséquilibré.



Implémentation d'un AVL

Rajouter un attribut à l'arbre binaire de recherche

- Sa hauteur

où

- Son facteur d'équilibrage

à mettre à jour à chaque modification (ajout ou suppression)

```
typedef struct noeud{
    int hauteur;
    TElement donnee;
    struct nœud *filsG, *filsD ;
} *AVL;
```

Primitives de bases :

Précondition : non estVideAVL(a)

Fonction donnee : AVL → TElement

Fonction donnee(a) → TElement

P.F a : AVL

Debut

Renvoyer a.donnee

Fin

Fonction filsGauche : AVL → AVL

Fonction filsGauche(a) → AVL

P.F a : AVL

Debut

Renvoyer a.filsG

Fin

Fonction filsDroit : AVL $\rightarrow$ AVLFonction filsDroit(a) $\rightarrow$ AVL

P.F a : AVL

Debut

Renvoyer a.filsD

Fin

Fonction hauteur : AVL $\rightarrow$ EntierFonction hauteur(a) $\rightarrow$ AVL

P.F a : AVL

Debut

Renvoyer a.hauteur

Fin

Algorithmes de bases :

Fonction initAVL : [] $\rightarrow$ AVLFonction initAVL() $\rightarrow$ AVL

Debut

Renvoyer NULL

Fin

Fonction estVideAVL : AVL $\rightarrow$ BooléenneFonction estVideAVL (a) $\rightarrow$ Booléenne

P.F a : AVL (E)

Debut

Renvoyer a=NULL

Fin

Fonction creatFeuille : TElementxEntier $\rightarrow$ AVLFonction creatFeuille(elt, tNoeud) $\rightarrow$ AVL

P.F elt : TElement (E)

tNoeud : Entier (E) // taille d'un noeud

Debut

f $\leftarrow$ allocMem(tNoeud)f.donnee $\leftarrow$ eltf.filsG $\leftarrow$ NULLf.filsD $\leftarrow$ NULLf.hauteur $\leftarrow$ 0

Renvoyer f

Fin

Fonction creatNoeud : TElementxEntierxAVLxAVL → AVL

Fonction creatNoeud(elt, tNoeud , fg, fd) → AVL

P.F elt : TElement (E)
 tNoeud : Entier (E)
 fg, fd : AVL

Debut

nd ← allocMem(tNoeud)

nd.donnee ← elt

nd.filsG ← fg

nd.filsD ← fd

nd.hauteur ← 1 + max(haut(fg) , haut(fd))

Renvoyer nd

Fin

```
int haut(Avl a) {
    return (a == null) ? -1 : hauteur(a);
}
```

Algorithmes de parcours (similaires aux arbres binaires)

1) Parcours en profondeur :

- parcoursPréfixe
- parcoursInfixe
- parcoursPostfixe

2) Parcours en largeur

Algorithme de recherche dans un AVL (↔ ABR)

Tri d'un tableau en passant par un AVL (↔ ABR)

Facteur d'Equilibre

Précondition : `non estVideAVL(a)`

Fonction factEquilibre : AVL → Entier

Fonction factEquilibre (a) → Entier

P.F a : AVL (E)

Debut

hd ← haut(filsDroit(a))

hg ← haut(filsGauche(a))

Renvoyer hd - hg

FIN

Rappel

```
int haut(Avl a) {
    return (a == null) ? -1 : hauteur(a);
}
```

Implémentation des rotations

Les opérations de rotations sont définies de la façon suivante :

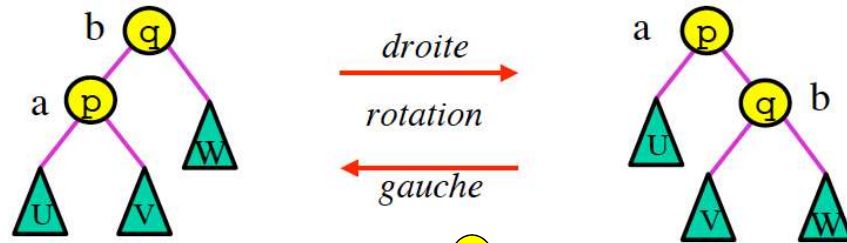
Précondition : `non estVideAVL(a)`

Fonction rotationG : Avl -> Avl

Fonction rotationD : Avl -> Avl

Fonction rotationGD : Avl -> Avl

Fonction rotationDG : Avl -> Avl

Fonction rotationG : AVL → AVL

Rotation Gauche : l'arbre de racine p est penché à droite et son sous arbre droit q est aussi penché à droite

Autrement dit : Si $\text{Déséquilibre}(p)=+2$ et $\text{Déséquilibre}(q)\leq +1$

Alors Rotation Gauche autour de p

Fonction rotationG(a) → AVL

P.F a : AVL (E)

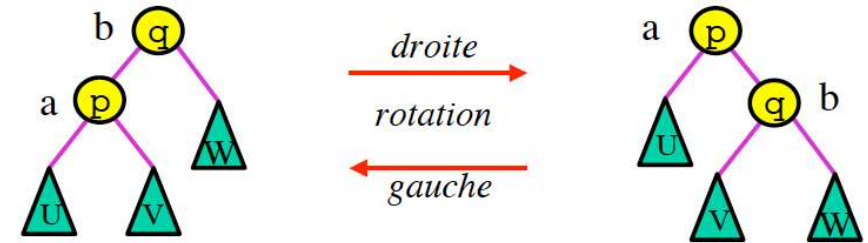
Debut

```

p ← a
u ← filsGauche(a)
q ← filsDroit(a)
v ← filsGauche(q)
w ← filsDroit(q)
q.filsG ← p
p.filsD ← v
p.hauteur ← 1+max(haut(u), haut(v))
q.hauteur ← 1+max(haut(p), haut(w))
renvoyer q

```

FIN

Fonction rotationD : AVL → AVL

Rotation Droite : l'arbre de racine q est penché à gauche et son sous arbre gauche p est aussi penché à gauche

Autrement dit : Si $\text{Déséquilibre}(q)=-2$ et $\text{Déséquilibre}(p)\leq 0$

Alors Rotation Droite autour de q

Fonction rotationD(a) → AVL

P.F a : AVL (E)

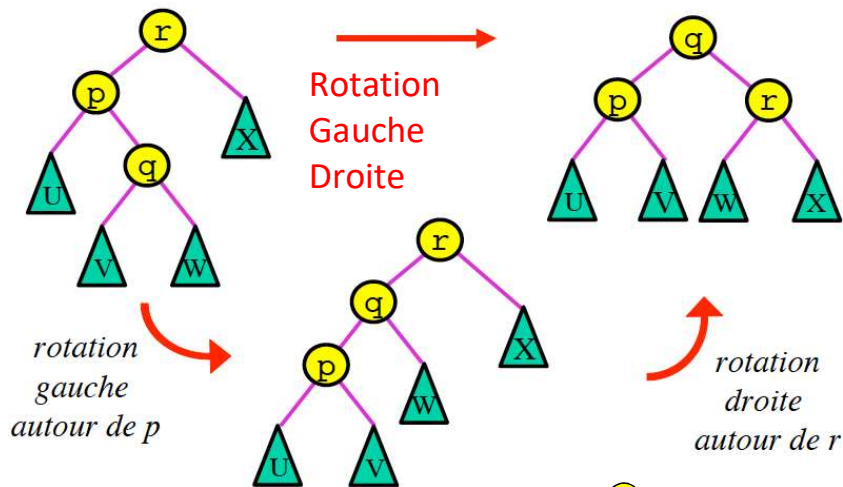
Debut

```

q ← a
p ← filsGauche(a)
w ← filsDroit(a)
u ← filsGauche(p)
v ← filsDroit(p)
p.filsD ← q
q.filsG ← v
q.hauteur ← 1+max(haut(v), haut(w))
p.hauteur ← 1+max(haut(u), haut(q))
renvoyer p

```

FIN

Double Rotation : Gauche+Droite

Rotation Gauche Droite : l'arbre de racine r est penché à gauche et son sous arbre gauche p est aussi penché à droite.

Autrement dit : Si Déséquilibre(r)=-2 et Déséquilibre(p)=+1

Alors Rotation Gauche Droite autour de r

Fonction rotationGD : AVL $\rightarrow$ AVL

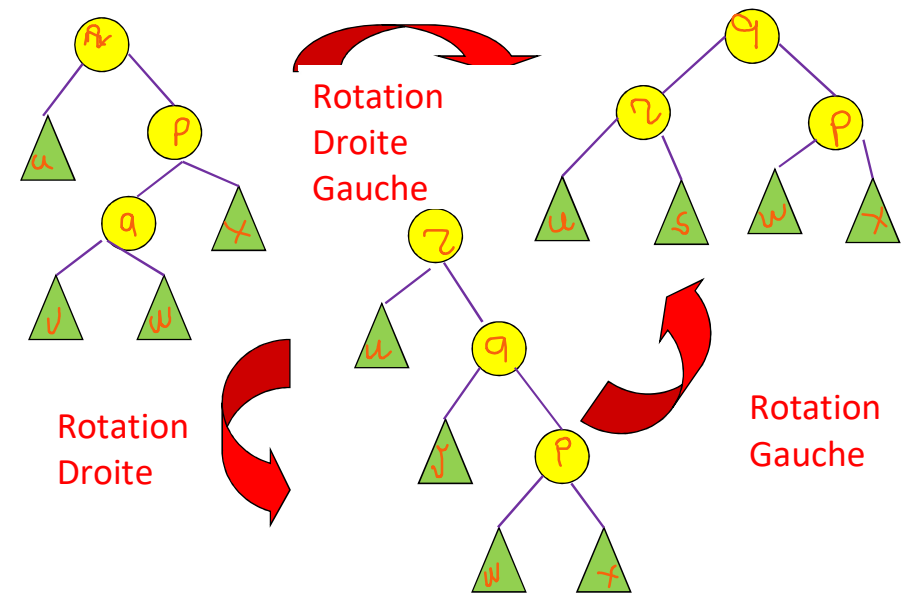
Fonction rotationGD(a) $\rightarrow$ AVL

P.F a : AVL (E)

Debut

a.filsG $\leftarrow$ rotationG(filsGauche(a))
renvoyer rotationD(a);

FIN

Double Rotation : Droite + Gauche

Rotation Droite Gauche : l'arbre de racine r est penché à droite et son sous arbre droit p est aussi penché à gauche

Autrement dit : Si Déséquilibre(r)=+2 et déséquilibre(p)=-1

Alors Rotation Droite Gauche autour de r

Fonction rotationDG : AVL $\rightarrow$ AVL

Fonction rotationDG (a) $\rightarrow$ AVL

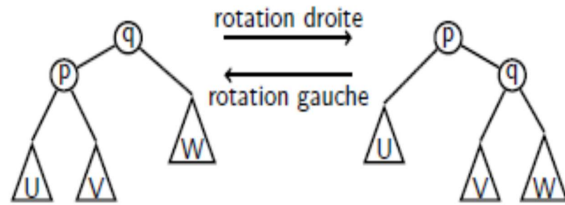
P.F a : AVL (E)

Debut

a.filsD $\leftarrow$ rotationD(filsDroit(a))
renvoyer rotationG(a);

FIN

Propriétés des rotations :



Propriété 1

- Les rotations préservent l'ordre infixe, et donc la propriété d'arbre binaire de recherche
- Par contre, elles modifient la hauteur de certains sous-arbres de 1 ou -1

Insertion dans un AVL

Principe :

Similaire à l'insertion dans un ABR + Équilibrage

Fonction inserAVL : TElement X Entier X AVL → AVL

Fonction inserAVL(elt, tNoeud, a) → AVL

P.F elt : TElement (E)

tNoeud : Entier (E) // taille d'un nœud

a : AVL (E)

Debut

Si estVideAVL(a) alors

renvoyer creatFeuille(tNoeud)

Fsi

si elt = donnee(a) alors

renvoyer a

Fsi

Si elt < donnee(a)

a.filsG ← inserAVL(elt, tNoeud, filsGauche(a))

Sinon

a.filsD ← inserAVL(elt, tNoeud, filsDroit(a))

Fsi

renvoyer equilibrage(a)

FIN

Fonction équilibrage

Précondition : non estVideAVL(a)

Fonction équilibrage : AVL → AVL

Fonction **equilibrage** (a) → AVL

P.F a : AVL (E)

Debut

```
fact ← factEquilibre(a)
```

```
si fact = -2 alors
```

```
    si factEquilibre(filsGauche(a)) <= 0 alors
```

```
        a ← rotationD(a)
```

```
    sinon
```

```
        a ← rotationGD(a)
```

```
    fsi
```

```
fsi
```

```
si fact = +2 alors
```

```
    si factEquilibre(filsDroit(a)) >= 0 alors
```

```
        a ← rotationG(a)
```

```
    sinon
```

```
        a ← rotationDG(a)
```

```
    fsi
```

```
fsi
```

```
a.hauteur ← 1+max(haut(filsGauche(a), haut(filsDroit(a)))
```

```
renvoyer a;
```

FIN

Note :

La complexité d'insertion dans un AVL est en $O(\log_2 n)$,
rééquilibrages compris

Suppression dans les AVL

Les opérations de suppression sont celles d'un arbre binaire de recherche dans lesquelles on ajoute la gestion du facteur d'équilibrage.

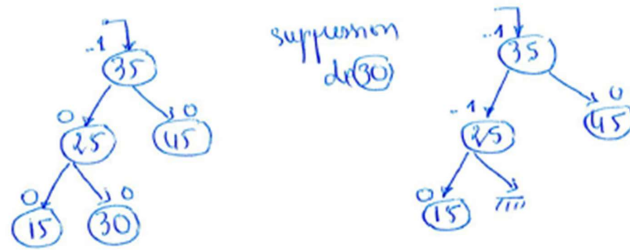
Après la suppression, il est possible que certains nœuds internes soient déséquilibrés.

On rééquilibre l'arbre tant qu'il y a un nœud déséquilibré sur le chemin de p à la racine, où p est le parent du nœud supprimé.

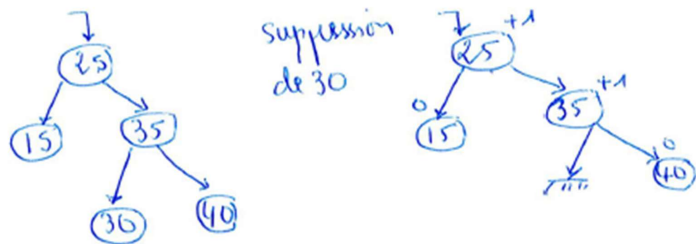
Propriété

Pour rééquilibrer un arbre AVL après une suppression, il faut jusqu'à h rotations simples ou doubles (h est la hauteur de l'arbre).

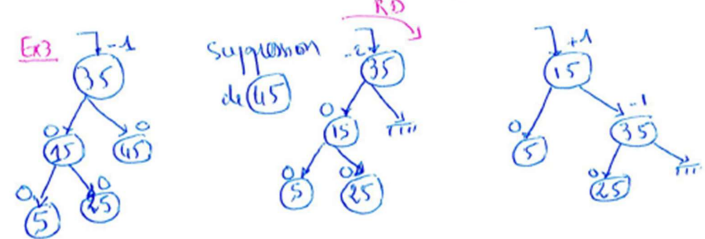
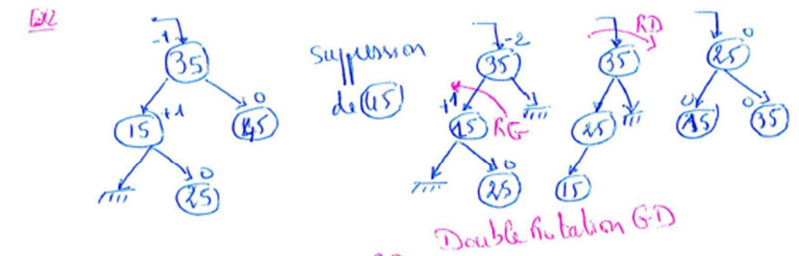
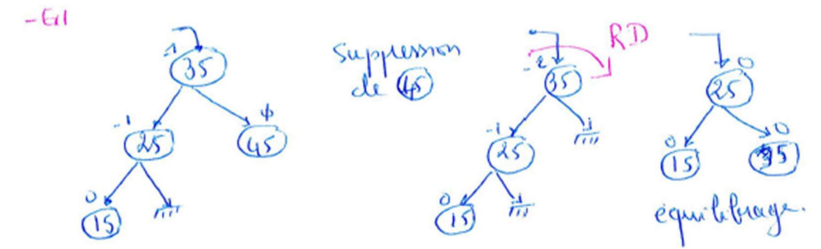
Exemples de suppressions



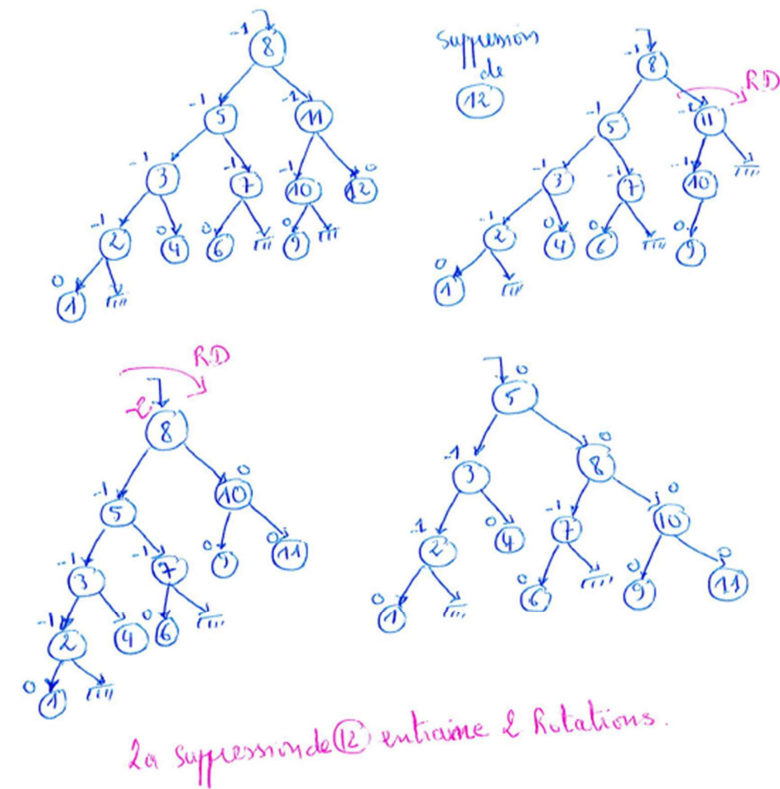
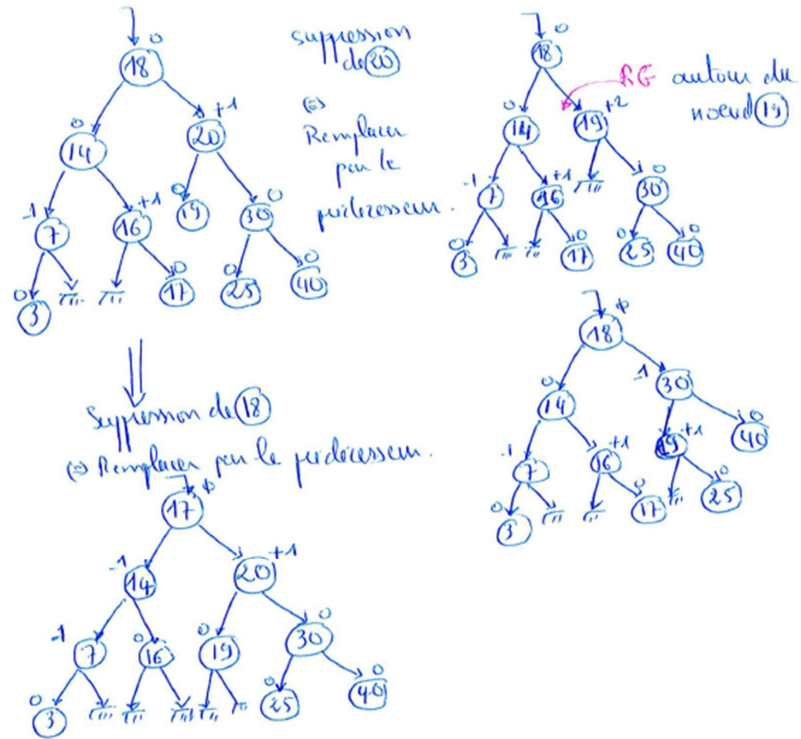
L'arbre reste équilibré après suppression.



L'arbre reste équilibré après suppression.



Suppression du sous-arbre droit



Algorithme de suppression d'un élément dans un AVL

Même principe que la suppression que dans un ABR + équilibrage

Fonction **suppEltAVL** : TElement X Arbre $\rightarrow$ Arbre

Fonction **suppEltAVL(elt, a) $\rightarrow$ Arbre**

P.F elt : TElement (E)

a : Arbre (E)

Debut

Si estVideA(a) alors

Renvoyer a

Fsi

da $\leftarrow$ donnee(a)

Si da = elt

Renvoyer suppRacineAVL(a)

Fsi

Si elt < da alors

a.filsG $\leftarrow$ suppEltAVL(elt, filsGauche(a))

alors

a.filsD $\leftarrow$ suppEltAVL(elt, filsDroit(a))

Fsi

Renvoyer equilibrer(a) // seul changement

Fin

Algorithme de suppression du nœud racine dans un AVL

Principe : est similaire à la suppression racine dans un ABR, il faut rajouter l'équilibrage.

Précondition : non est Vide (a)

Fonction **suppRacineAVL(a) $\rightarrow$ Arbre**

P.F a : Arbre (E)

Debut

r = a;

fg = filsGauche(a)

fd = filsDroit(a)

si estFeuille(a) alors

a $\leftarrow$ initA()

libMem(r)

renvoyer a

fsi

si (estVideA(fg))

a $\leftarrow$ fd

libMem(r)

renvoyer equilibrer(a)

fsi

si (estVideA(fd)) alors

a $\leftarrow$ fg

libMem(r)

renvoyer equilibrer(a)

fsi

a.donnee $\leftarrow$ suppMaxAVL(fg);

renvoyer equilibrer(a) //seul changement

Fsi

Fin

Algorithme de suppression du maximum dans un AVL

Principe : est similaire à la suppression du maximum dans un ABR, il faut rajouter l'équilibrage.

Précondition : non est Vide (a)

Cet algorithme supprime le nœud qui contient le max dans un ABR et il retourne la donnée du maximum.

Fonction supMaxAVL: Arbre → Arbre X TElement

Fonction supMaxAVL(a)→TElement

P.F a : Arbre (E)

Debut

fd ← filsDroit(a)

Si estVideA(fd) alors

aa ← a

eMax ← donnee(a)

a ← filsGauche(a)

libMem(aa)

renvoyer eMax

Sinon

renvoyer supMaxAVL(fd)

Fsi

FIN

Complexité

Complexité

- La complexité de chacune des procédures est majorée par la hauteur de l'arbre.
- Remarque: les rotations se font en temps constant.
- Bilan:
 - ▶ insérer, rechercher, supprimer, maximum, minimum s'effectuent en temps $O(\log n)$.
- Remarque: rééquilibrer un arbre
 - ▶ après une insertion, en fait une seule rotation, ou double rotation suffit.
 - ▶ après une suppression, il peut y avoir jusqu'à h rotations ou double rotations.

Références

<https://www.enseignement.polytechnique.fr/profs/informatique/Jean-Eric.Pin/PDF/Amphi9.pdf>

http://igm.univ-mlv.fr/~mac/ENS/DOC/arbravl_7.pdf